How DQC works

DQC is a set of Airflow operators to help identify and alert Data Quality Issues. You can plug it in any of your Airflow DAGs. It will execute SQL query and check if the data is expected with your own configuration. It supports notification on email/Slack/PagerDuty.

In general you have two options to use DQC:

- Configure entirely by you self using **DQFreeFormSQLCheckOperator**
- Use existing operators which are already wrapped by us

1. Import module

On legacy Airflow (Clroot):

```
from data_sci.airflow_operators.tahoe.functional_operator import
DQFreeFormSQLCheckOperator, DQValueCheckOperator
```

On Airflow 2.0 (data-pipelines)

```
from src.contrib.data_sci.data_quality_operator import DQFreeFormSQLCheckOperator,
DQValueCheckOperator
```

2. Choose the operator

You can have a glance at the section “Available Operators” below. Each operator is designed to solve specific problems. Of course you can alway use **DQFreeFormSQLCheckOperator** to make the most flexible check. You can refer to this doc to find the usage of each operator for certain data quality issues.

3. Provide SQL and threshold

- For **DQFreeFormSQLCheckOperator**, you need to provide SQL query and a ***validate_func*** which takes the query result (list<dict>) as input and returns `pass_flag`(boolean; if the test passes/fails) and `log_in_detail`(string; this message will be in the notification/leave it as empty string if you don’t want to add any message). You need to implement the rules inside your validate_func.
- For **DQValueCheckOperator**, you need to configure some input variables to let the operator ensemble SQL query and validate_func. You can take the demo below as an example. The **value** inside column_map is a Python list representing the expected value range; for example, for the aggregated value count(tracking_id), we expect its value to be in [1, ∞). Anything outside such range will yield the alert.

```
DQValueCheckOperator(
  task_id='dqc_value_check',
  dag=dag,
  db='wishpost',
  table='wishpost_order_clean_full_cycle',
  column_map={'tracking_id_cnt': {'expression': 'count(tracking_id)',
                                  'value': [1, float('inf')]},
              'shipping_country_cnt': {'expression': 'count(distinct shipping_country)',
                                       'value': [1, 5]}

              },
  dimension=['dt'],
  partition_condition="""dt = '{{ds}}'""",
  extra_condition='order_created_time > 1648778400',
  dependencies=[dqc_hist_task]
)
```

We have in-class documentation for each operator in case you want to know how to use them.
More details can also be found in [the design doc]
Contact us if you have any questions :) Jialiang Cui Raymond Xu Evelyn Bai

Available Operators

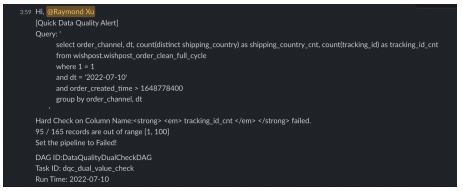
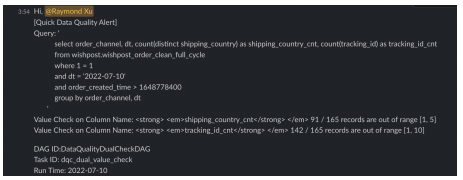
No.	Operator Name (Inherits From)	Usage Example	Details
1	**DQFreeFormSQLCheckOperator (NaivePythonOperator)**	dqc_free_form = DQFreeFormSQLCheckOperator(task_id='dqc_free_sql_check', dag=dag, sql_query=""" select is_registered, count(1) as cnt from wishpost.dim_order_channel_cn_mongo where is_registered is not null group by 1 """, validate_func=demo_validate_func, slack_alert=True, slack_channel='#dqc-slack-alert-test', slack_recipients=['jcui', 'raxu', 'ebai'] , failed_pipeline=False)	- ***sql_query*** is provided by user - ***validate_func*** takes the `result` of sql_query as input, returns `pass_flag, log_in_detail` - when `pass_flag` return False, alert messages will be sent. Default alert method is email. Recipients use dag owner as default, and can also be overridden as a list of string E.g. ['name1', 'name2', 'name3'] - Alert messages can also be sent via slack. Simply set ***slack_alert*** to True, specify ***slack_channel***, and ***slack_recipients***(dag owner as default) **There are some prerequisites for receiving Slack notifications, see below for details**

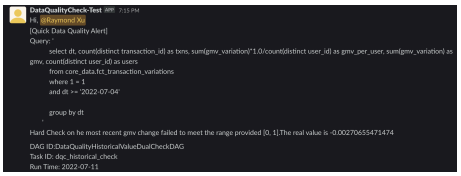
			- ***failed_pipeline*** indicates whether the rest of the tasks will proceed when the DQC task fails. If set to ***True,*** the rest tasks will not proceed, and the pipeline will be set to be failed.
2	**DQValueCheckOperator (DQFreeFormSQLCheckOperator)**	dqc_value_task = DQValueCheckOperator(task_id='dqc_value_check', dag=dag, db='wishpost', table='wishpost_order_clean_full_cycle', column_map={'tracking_id_cnt': {'expression': 'count(tracking_id)', 'value': [1, float('inf')]}, 'shipping_country_cnt': {'expression': 'count(distinct shipping_country)', 'value': [1, 5]} }, dimension=['dt'], partition_condition="""dt = '{{ds}}'""", extra_condition='order_created_time > 1648778400', dependencies=[dqc_hist_task])	- sql_query is generated based on ***column_map, dimension***, and any ***conditions*** provided in the Operator Params. The example on the left will generate the sql as this: ‘ select order_channel, dt, count(distinct shipping_country) as shipping_country_cnt, count(tracking_id) as tracking_id_cnt from wishpost.wishpost_order_clean_full_cycle where 1 = 1 and dt = '2022-07-10' and order_created_time > 1648778400 group by order_channel, dt ’ -
3	**DQDuplicateCheckOperator (DQFreeFormSQLCheckOperator)**	dqc_duplicate_task = DQDuplicateCheckOperator(task_id='dqc_duplicate_check', dag=dag, db='wishpost', table='wishpost_order_clean_full_cycle', column='carrier_pickup_state', partition_condition="""dt = '{{ds}}'""", extra_condition='order_created_time > 1648778400',)	-sql_query is generated based on the column and any ***conditions*** provided in the Operator Params. Check if there’s duplicate key values in a column. The example on the left will generate the sql as this: select cnt_duplicate_value, count(1) as cnt_record from (select carrier_pickup_state, count(1) as cnt_duplicate_value from wishpost.wishpost_order_clean_full_cycle where 1 = 1 and dt = '2022-06-27' and order_created_time > 1648778400 group by 1) t group by 1
4	**DQHistoricalValueCheckOperator**	dqc_hist_task =	- sql_query is generated based on

	(DQValueCheckOperator)	DQHistoricalValueCheckOperator(task_id='dqc_historical_check', dag=dag, db='core_data', table='fct_transaction_variations', column_map={'gmv': {'expression': 'sum(gmv_variation)', 'value': [-0.03, 0.03]}, 'txns': {'expression': 'count(distinct transaction_id)', 'value': []}, 'users': {'expression': 'count(distinct user_id)', 'value': []}, 'gmv_per_user': {'expression': 'sum(gmv_variation)*1.0/count(distinct user_id)', 'value': []}, }, dimension=['dt'], partition_condition="""dt >= '{{macros.ds_add(ds, -7)}}' """, dependencies=[dqc_free_form], fail_pipeline=False)	***column_map***, ***dimension***, and any ***conditions*** provided in the Operator Params. The example on the left will generate the sql as this: select dt, count(distinct transaction_id) as txns, sum(gmv_variation)*1.0/count(distinct user_id) as gmv_per_user, sum(gmv_variation) as gmv, count(distinct user_id) as users from core_data.fct_transaction_variations where 1 = 1 and dt >= '2022-06-07' group by d **SQL Query** select dt, count(distinct transaction_id) as txns, sum(gmv_ gmv_per_user, sum(gmv_variation) as gmv, count(distinc core_data.fct_transaction_variations where 1 = 1 and dt >= The most recent users not between the range with mean 115950.491578].The real value is 104033.0 **Data Summary** txns dt gmv_per_user gmv users 124859 2022-06-13 28.1230919997 2925729.63 104033 136629 2022-06-10 29.1040253252 3279849.03 112694 137426 2022-06-12 29.2954576349 3336137.42 113879 132734 2022-06-08 29.3660459388 3230734.91 110016 137193 2022-06-09 29.0044528459 3279707.51 113076 132468 2022-06-11 29.0342459275 3161887.45 108902 136630 2022-06-07 29.0746375761 3289533.57 113141 t
5	**DQRegexMatchOperator** (DQFreeFormSQLCheckOperator)	dqc_regex_check = DQRegexMatchOperator(task_id='dqc_regex_check', dag=dag, db='wishpost', table='wishpost_order_clean_full_cycle', column='tracking_id', regex='(^W.*FPL$)|^UD', partition_condition="""dt = '{{ds}}'""", extra_condition='order_created_time > 1648778400')	-sql query is generated based on user-defined target ***column***, target ***regex expression***, together with any ***conditions*** provided in the Operator Params. -check if all values in the target column match with the regex expression. The example on the left will generate the sql as this: select count(tracking_id) as cnt from wishpost.wishpost_order_clean_full_cycle where 1=1 and not regexp_like(tracking_id, '(^W.*FPL$)|^UD')

			and dt = '2022-06-22' and order_created_time > 1648778400 The alert email be like: **SQL Query** select count(tracking_id) as cnt from wishpost.wishpost_order_clean_full_cycle where 1=1 and not regexp_like(tracking_id, '(^W.*FPL$)\|^UD') and dt = '2022-06-22' and order_created_time > 1648778400 **DQ Check on tracking_id fails: 1147452 values failed to match (^W.*FPL$)|^UD** **Data Summary** cnt 1147452

6	**DQValueInSetCheckOperator (DQFreeFormSQLCheckOperator)**	dqc_value_task5 = DQValueInSetCheckOperator(
		task_id = 'dqc_value_equality_check_2',
		dag=dag,
		db='wishpost',
		table='wishpost_order_clean_full_cycle',
		column_map={
		'registered': {
		'expression': 'registered',
		'value': ['true']
		},
		'cancel_src': {
		'expression': 'cancel_source',
		'value': [1]
		}
		},
		partition_condition="""dt = '{{ds}}'""",
		extra_condition='order_created_time > 1648778400',
		)
			-User needs to specify a ***column_map*** where the key is the alias of selected columns or aggregation expressions, and other ***conditions*** used to generate the sql.
			
			-This operator solves data issues under condition of:
			• Check if all values in target column(s) are in a predefined set
			• Check if aggregation of target column(s) is in a predefined set
			It can also solve value equality data issues by setting the predefined set to a specific number.
			
			The example on the left will generate a sql like this:
			select distinct registered, cancel_src
			from
			(select registered as registered, cancel_source as cancel_src
			from wishpost.wishpost_order_clean_full_cycle
			where 1 = 1
			and dt = '2022-06-22'
			and order_created_time > 1648778400
			
			) as t
			order by 1
			
			-In the alert email, the alert log is meaningful, but data summary will be the cartesian product of elements in two columns.

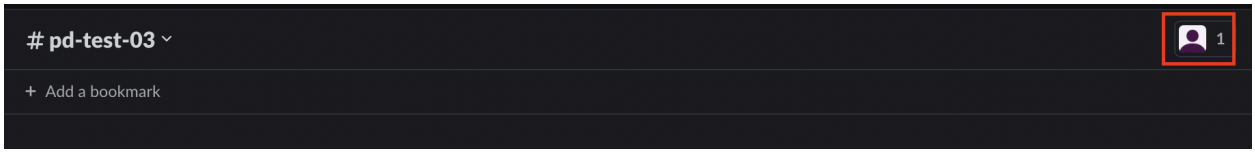
			SQL Query select distinct registered, cancel_src from (select registered as registered, cancel_source as cancel_src from wishpost.wishpost_order_clean_full_cycle where 1 = 1 and dt = '2022-06-21' and order_created_time > 1648778400) as t order by 1 **Column registered has invalid values that fail the DQC: [u'false']** **Column cancel_src has invalid values that fail the DQC: [None, 2]** **Data Summary** registered cancel_src false 1 false None false 2 true 2 true 1 true None
7	**DQDualValueCheckOperator (DQFreeFormSQLCheckOperator)**	dqc_dual_value_task = DQDualValueCheckOperator(task_id='dqc_value_check', dag=dag, db='wishpost', table='wishpost_order_clean_full_cycle', column_map={'tracking_id_cnt': {'expression': 'count(tracking_id)', 'value': [1, 10000], 'hard_check_value': [0, float('inf')], }, 'shipping_country_cnt': {'expression': 'count(distinct shipping_country)', 'value': [1, 5]} }, dimension=['dt'], partition_condition="""dt = '{{ds}}'""", extra_condition='order_created_time > 1648778400', dependencies=[dqc_hist_task])	- allow dual value criteria check on columns - Operator checks the ***'hard_check_value'*** first, if pass_dqc fails, the task will terminate and pipeline will be set to fail. - If ***'hard_value_check'*** passed and ***'value'*** doesn't, alert messages will still be sent via email or slack, but the pipeline will continue. -  
8	**DQDualHistoricalValueCheckOperator (DQValueCheckOperator)**	dqc_hist_task = DQDualHistoricalValueCheckOperator(task_id='dqc_dual_historical_check', dag=dag, db='core_data', table='fct_transaction_variations', column_map={'gmv': {'expression': 'sum(gmv_variation)', 'value': [-0.03, 0.03], 'hard_check_value': [-1, 1] }, 'txns': {'expression': 'count(distinct transaction_id)',	- allow dual historical value criteria check on columns - if ***'hard_check_value'*** is provided, Operator checks it first and fail the task & pipeline if value out of range

		'value': []}, 'users': {'expression': 'count(distinct user_id)', 'value': []}, 'gmv_per_user': {'expression': 'sum(gmv_variation)*1.0/count(distinct user_id)', 'value': []}, }, dimension=['dt'], partition_condition="""dt >= '{{macros.ds_add(ds, -7)}}' """, dependencies=[dqc_free_form], fail_pipeline=False)	

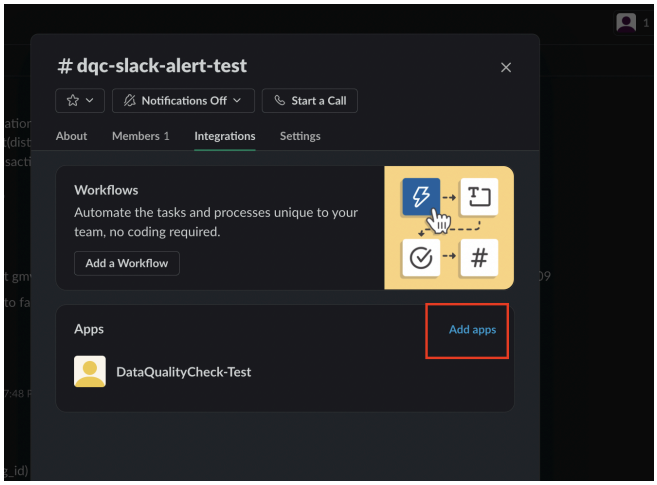
Set up Slack Notification

Steps for integrating Default SlackBot App [DataQualityCheck-Test]

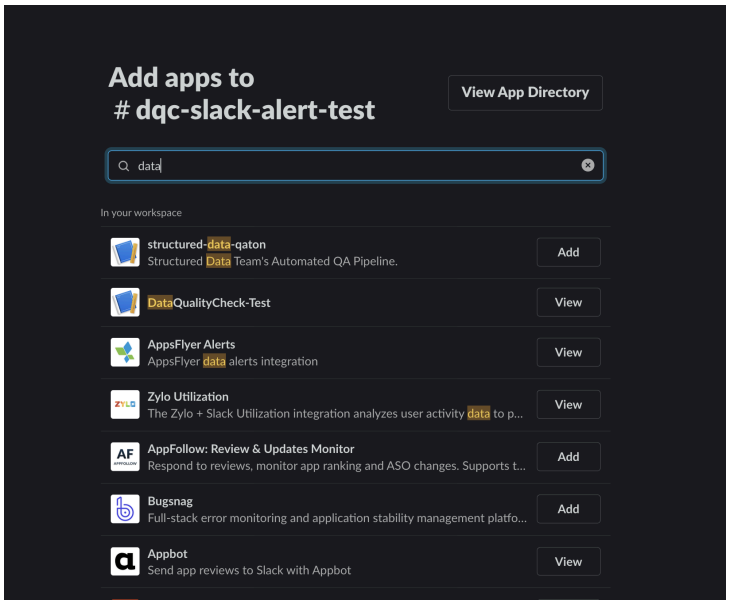
- Choose a channel that you want to receive your alert message
- Click View All Members



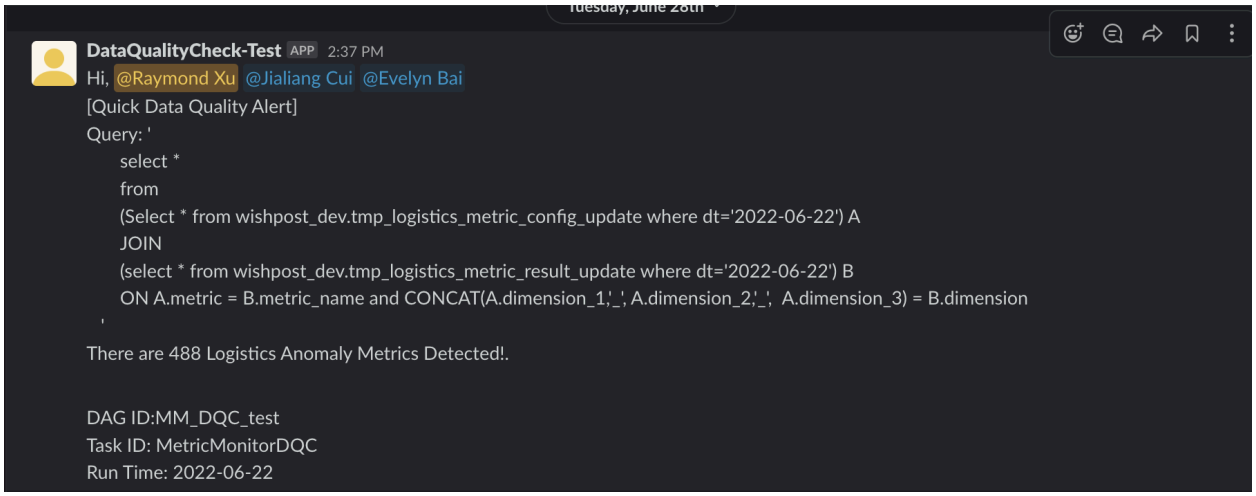
Under '*Integration*" , click 'Add apps'



Search for app named *'DataQualityCheck-Test' , click add*

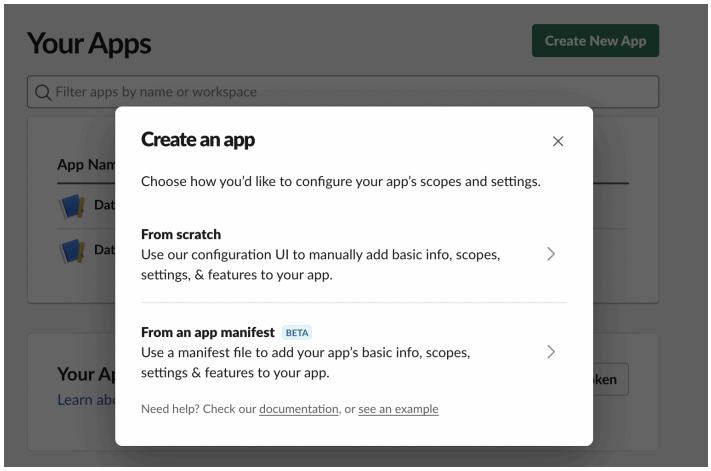


- Now you are good to go. Specify the correct slack channel name in your DQC Operators and you will receive your alert messages when needed

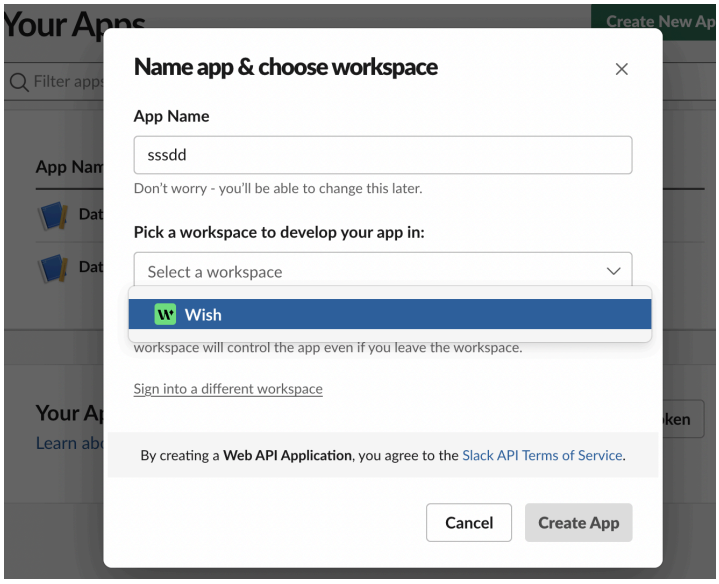


Steps for Creating your own SlackBot App

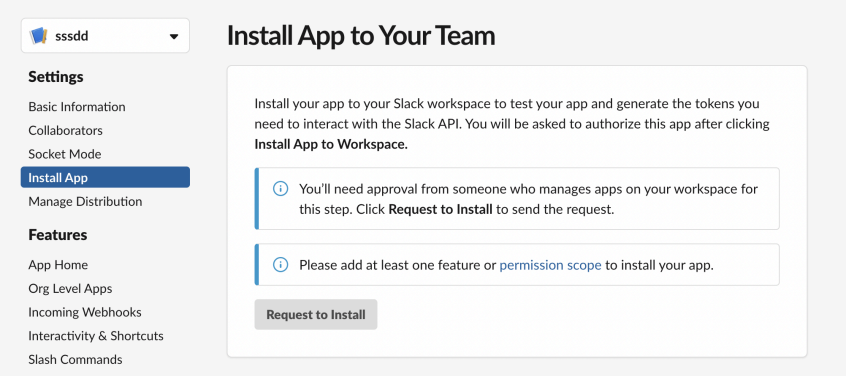
- Go to https://api.slack.com/apps?new_app=1
- Click *'Create New App'*



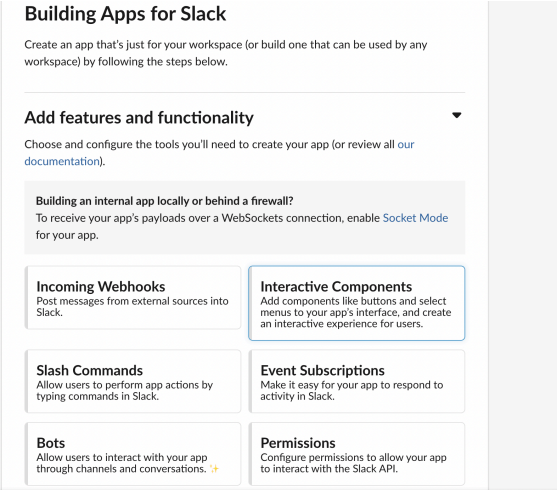
- Specify app name and workspace



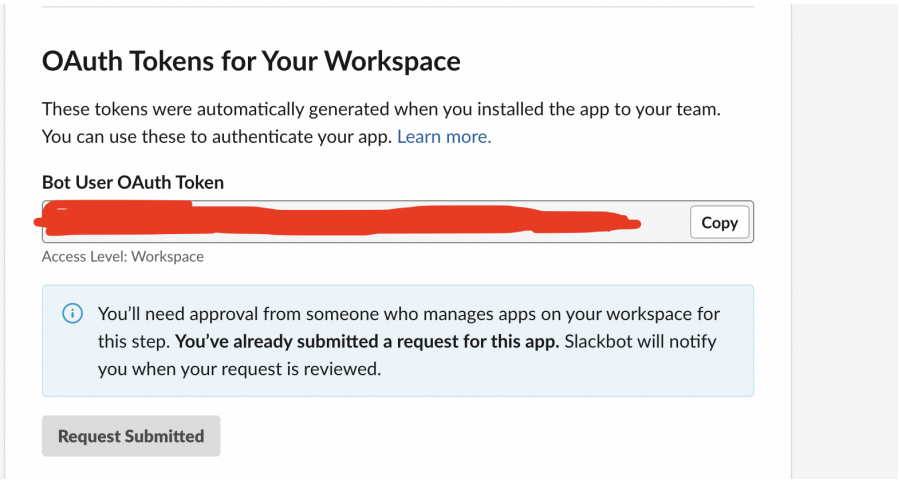
- **Request Installation of your app (takes some time to get approved)**



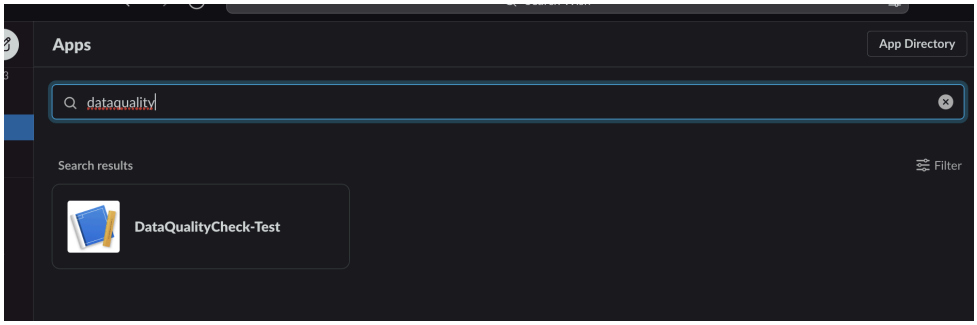
- **Add all necessary features and permissions of your app**



- **After a request is approved, an Oauth Token will be generated. Use it as the token for authenticating your app.(e.g. Slack_token parameter of DQC Operators)**



- **When successfully installed, you can search your app under your workspace.**



Integrate PagerDuty

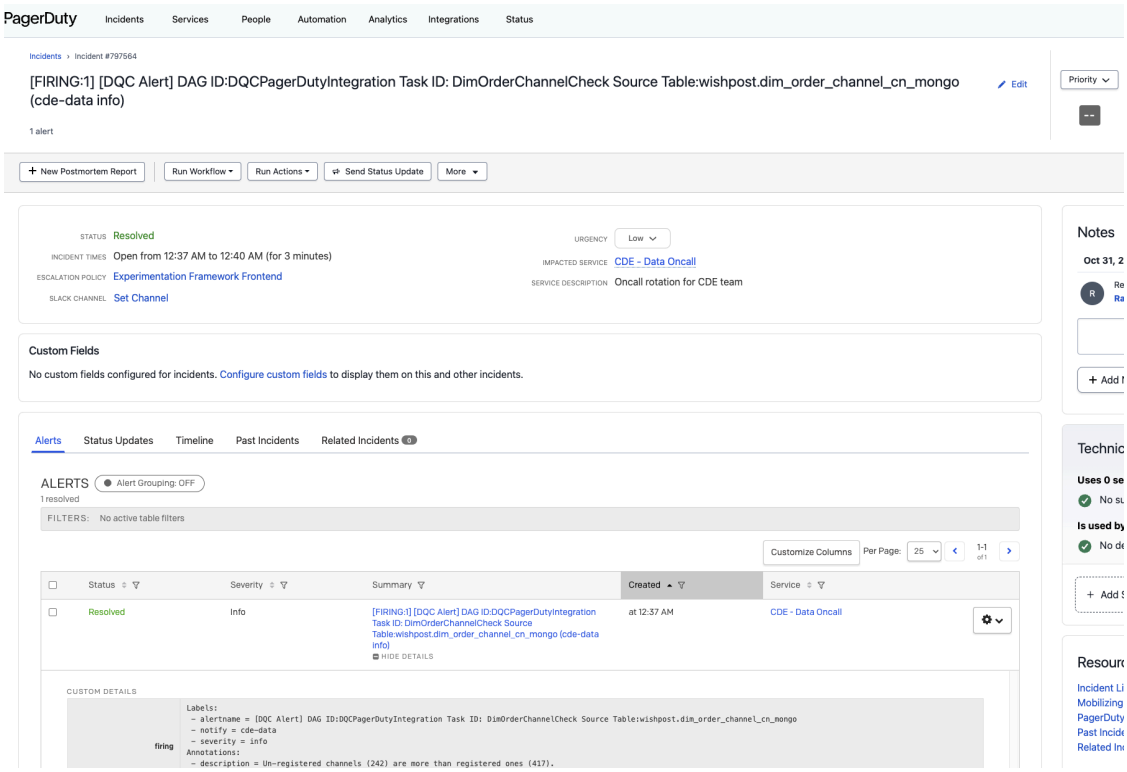
It is easy to trigger PagerDuty alert, all you need to do is set up the following parameters:

- `pagerduty_alert (please set this to True)`
- `pagerduty_severity (like 'Info' or 'Warning' or etc)`
- `pagerduty_notify (the group you are in such as 'cde-data')`

Attention! Only DQC on Airflow 2.0 supports PagerDuty alert!

```
dqc_free_form = DQFreeFormSQLCheckOperator(
    task_id='DimOrderChannelCheck',
    dag=dag,
    sql_query="""
           select is_registered, count(1) as cnt
           from wishpost.dim_order_channel_cn_mongo
           where is_registered is not null
           group by 1
    """,
    validate_func=demo_validate_func,
    slack_alert=True,
    slack_channel='#dqc-slack-alert-test',
    slack_token='XXXXX',
    check_type='DQCFreeFormCheck',
    pagerduty_alert=True,
    pagerduty_severity='Info',
    pagerduty_notify='cde-data',
    source_table='wishpost.dim_order_channel_cn_mongo'

)
```


PagerDuty
Incidents
Services
People
Automation
Analytics
Integrations
Status
Incidents › Incident #797564
[FIRING:1] [DQC Alert] DAG ID:DQCPagerDutyIntegration Task ID: DimOrderChannelCheck Source Table:wishpost.dim_order_channel_cn_mongo (cde-data info)
Edit
Priority
1 alert
New Postmortem Report
Run Workflow
Run Actions
Send Status Update
More
STATUS Resolved
INCIDENT TIMES Open from 12:37 AM to 12:40 AM (for 3 minutes)
ESCALATION POLICY Experimentation Framework Frontend
SLACK CHANNEL Set Channel
URGENCY Low
IMPACTED SERVICE CDE - Data Oncall
SERVICE DESCRIPTION Oncall rotation for CDE team
Custom Fields
No custom fields configured for incidents. Configure custom fields to display them on this and other incidents.
Alerts
Status Updates
Timeline
Past Incidents
Related Incidents 0
ALERTS
Alert Grouping: OFF
1 resolved
FILTERS: No active table filters
Customize Columns
Per Page: 25
1-1 of 1
Status
Severity
Summary
Created
Service
Resolved
Info
[FIRING:1] [DQC Alert] DAG ID:DQCPagerDutyIntegration Task ID: DimOrderChannelCheck Source Table:wishpost.dim_order_channel_cn_mongo (cde-data info)
HIDE DETAILS
at 12:37 AM
CDE - Data Oncall
CUSTOM DETAILS
firing
Labels:
- alertname = [DQC Alert] DAG ID:DQCPagerDutyIntegration Task ID: DimOrderChannelCheck Source Table:wishpost.dim_order_channel_cn_mongo
- notify = cde-data
- severity = info
Annotations:
- description = Un-registered channels (242) are more than registered ones (417).
Notes
Technic
Resourc